ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse OpenAPI 3.0 schema definitions and generate dynamic HTTP mock endpoints returning randomized schema-compliant JSON payloads.	High	Pass: Mock endpoint responds with correct JSON schema types; Fail: Invalid endpoint path returns 200 OK.	Ensures developers can quickly test APIs without writing backend logic.
FR-002	Functional	The system shall allow users to configure simulated response latency for each endpoint.	Medium	Pass: Latency matches user configuration within ±10 ms; Fail: Latency deviation exceeds threshold.	Helps QA engineers simulate real-world network conditions.
FR-003	Functional	The system shall validate incoming requests against OpenAPI schema definitions and return appropriate error codes for invalid requests.	High	Pass: Invalid requests return 400/422 error codes; Fail: Invalid requests return 200 OK.	Guarantees schema compliance and prevents false positives in testing.
FR-004	Functional	The system shall provide a user interface (CLI or web dashboard) to upload OpenAPI specs and manage mock endpoints.	Medium	Pass: User can upload spec and see endpoints listed; Fail: Upload fails without error message.	Improves usability and accessibility for developers and testers.

FR-005	Functional	The system shall support multiple concurrent OpenAPI specifications, enabling developers to mock multiple APIs simultaneously.	Low	Pass: Multiple specs can be uploaded and mocked; Fail: Only one spec is supported at a time.	Enhances productivity when working with microservices or multiple APIs.
NFR-001	Performance & Security	The mock server engine must sustain 1,000 mock API requests per second with user-configured simulated latency accurate within ±10 ms.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load.	Ensures scalability and reliability under stress conditions.
NFR-002	Reliability & Maintainability	The system must provide detailed logging and error reporting, ensuring 99.9% uptime and easy debugging for developers.	Medium	Pass: Logs capture errors with timestamps; Fail: Errors occur without traceability.	Improves maintainability and supports long-term system stability.

Flow Specification:

Preconditions

- User has a valid OpenAPI 3.0 YAML/JSON file.
- System is running and accessible.

Postconditions

- Mock endpoints are generated and available for testing.

Main Success Scenario

1. API Developer selects “Upload Spec” option.
2. System prompts for file selection.
3. Developer uploads valid OpenAPI file.
4. System parses schema definitions.
5. System generates mock endpoints with randomized schema-compliant payloads.
6. Confirmation message is displayed with endpoint list.
7. Use case ends successfully.

Alternate Flow (Invalid Spec)

If uploaded file is invalid or not OpenAPI 3.0 compliant:

- System displays error message with details.
- Developer is prompted to re-upload a corrected file.
- If error persists after 2 attempts, process is cancelled.